# VULNERABILITY ASSESSMENT REPORT

## Insecure Direct Object Reference (IDOR) in the Purchase / OTP Validation Workflow

Target Application: Yavuzlar Vulnerable Web Lab ("Free Purchase" Module)

by: Reinaldy Thendean

Date: July 2026

*Classification: Portfolio / Educational Documentation*

Disclaimer: This assessment was performed against "Yavuzlar", a deliberately vulnerable web application built for penetration testing training and self-study purposes. All testing was conducted in an isolated lab environment with explicit permission to attack; no real users, production systems, or real financial data were involved. This document is published solely to demonstrate methodology and technical understanding for portfolio purposes.

# Report Information

| | |
|---|---|
| **Report Title** | IDOR on Purchase / OTP Validation Workflow — "Free Purchase" Vulnerability |
| **Target** | Yavuzlar Vulnerable Web Application (training vulnerable lab) |
| **Tester** | Reinaldy Thendean (Undergraduate Student, Web Application Security) |
| **Testing Type** | Manual black-box web application penetration testing |
| **Tools Used** | Burp Suite (Proxy/Intercept), Web Browser Developer Tools, Base64 decoder |
| **Vulnerability Class** | Insecure Direct Object Reference (IDOR) / Broken Access Control leading to Business Logic Bypass |
| **CWE Reference** | CWE-639 (Authorization Bypass Through User-Controlled Key), CWE-841 (Improper Enforcement of Behavioral Workflow), CWE-284 (Improper Access Control) |
| **CVSS v3.1 Base Score** | 7.5 (High) |
| **CVSS v3.1 Vector** | CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N |

# 1. Executive Summary

During a manual security assessment of the "Free Purchase" module in the Yavuzlar training lab, an Insecure Direct Object Reference (IDOR) vulnerability was identified in the transaction/OTP validation workflow. The flaw allows an authenticated user to bypass the server-side account-balance check that is normally enforced when a purchase is initiated, and to complete a purchase for an item priced above their available balance.

The root cause is that the balance/funds validation is only performed at the initial "Buy Item" step on the client-facing flow, while the actual purchase-completion endpoint (the OTP/transaction validation page) can be accessed directly, out of sequence, using a static and predictable reference value. Because this endpoint does not verify that a legitimate, balance-checked transaction was actually created beforehand, an attacker can skip the balance check entirely and finalize a purchase they should not be able to afford.

This finding was rated High severity (CVSS v3.1 base score 7.5) because it does not expose confidential data or affect availability, but it directly compromises the integrity of the application's core purchasing/business logic — allowing unauthorized transactions to complete.

# 2. Scope

- Application: Yavuzlar vulnerable web application (intentionally vulnerable lab designed for pentest training).
- Module in scope: "Free Purchase" / shopping and checkout flow, specifically the item purchase and OTP validation process.
- Test type: Manual dynamic testing (DAST) with proxy interception; no automated scanners were used.

- Authentication: Testing was performed as a standard authenticated user with a limited starting balance.

# 3. Methodology

The assessment followed a standard manual black-box testing approach:

**1.** Explore the normal application workflow (baseline behavior) before attempting to manipulate it.

**2.** Intercept all HTTP requests and responses using Burp Suite to understand the parameters involved in the transaction flow.

**3.** Decode and analyze any encoded parameters (e.g., Base64) to determine what data is being passed between client and server.

**4.** Repeat the transaction flow to identify which values are static/predictable versus dynamically generated per transaction.

**5.** Attempt to manipulate parameter values directly (e.g., balance) to test server-side validation.

**6.** Attempt to access internal workflow steps out of order or directly via URL, to test whether server-side state/authorization enforcement exists between steps.

**7.** Validate impact by attempting to purchase an item priced above the account's actual balance.

# 4. Vulnerability Analysis & Proof of Concept

## 4.1 Baseline Purchase Flow

The first step was to observe a normal, legitimate purchase to understand the expected workflow. The tester attempted to purchase an item called "Thief Cat", priced at $100.

Step 1 — Attempt a normal purchase of the "Thief Cat" item ($100).

Step 2 — Proceed to the shopping cart.

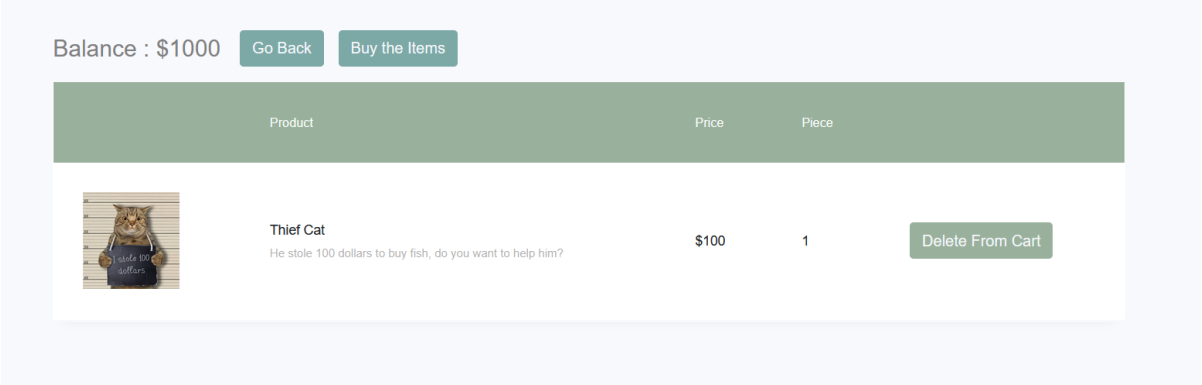


*Figure 1. Shopping cart page before checkout.*

Step 3 — Click "Buy Item" to initiate checkout. At this point, the application prompts the user to enter an OTP (One-Time Password) that is delivered via an in-app message box.

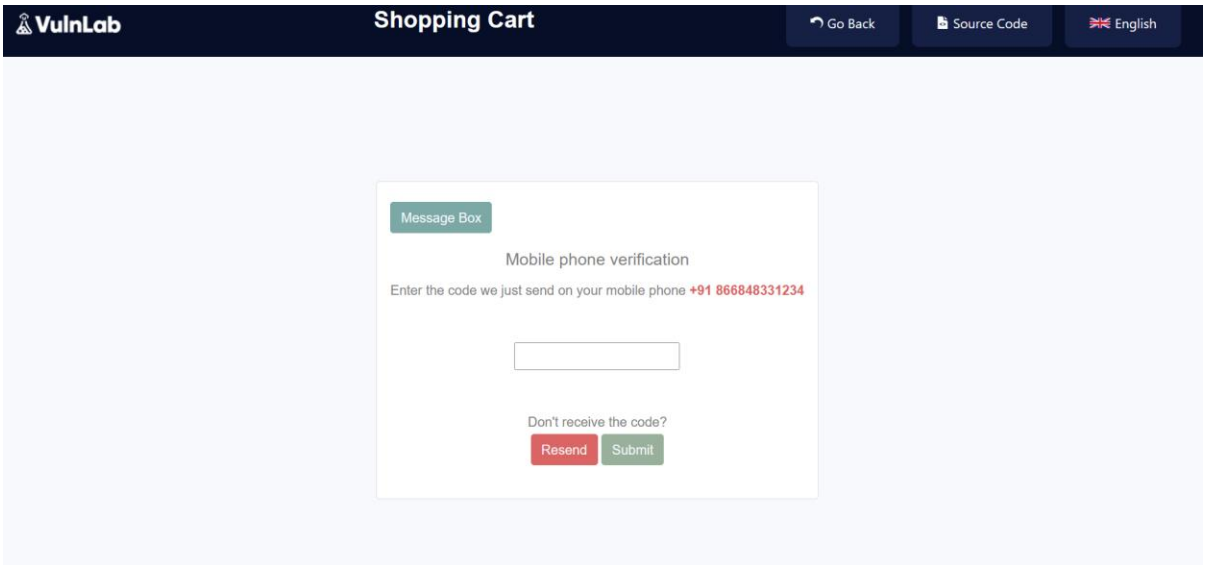


*Figure 2. Clicking "Buy Item" triggers the checkout/OTP request.*

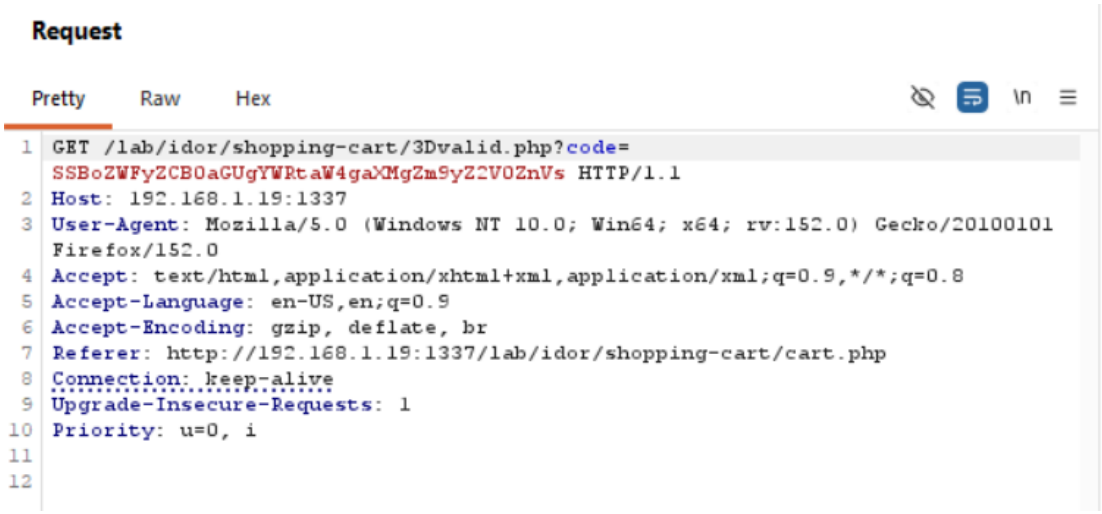


*Figure 3. OTP prompt after initiating the purchase.*

Inspecting the request sent to the OTP validation page in Burp Suite revealed a parameter named "code", whose value is Base64-encoded. Decoding this value produced the plaintext string "I heard the admin is forgetful" — a hard-coded / joke value rather than a legitimate, randomly generated per-transaction token.

## 4.2 Submitting the OTP and Observing the "hesoyam" Parameter

Step 4 — The tester opened the in-app message box, retrieved the OTP, submitted it, and observed the resulting request in Burp Suite.

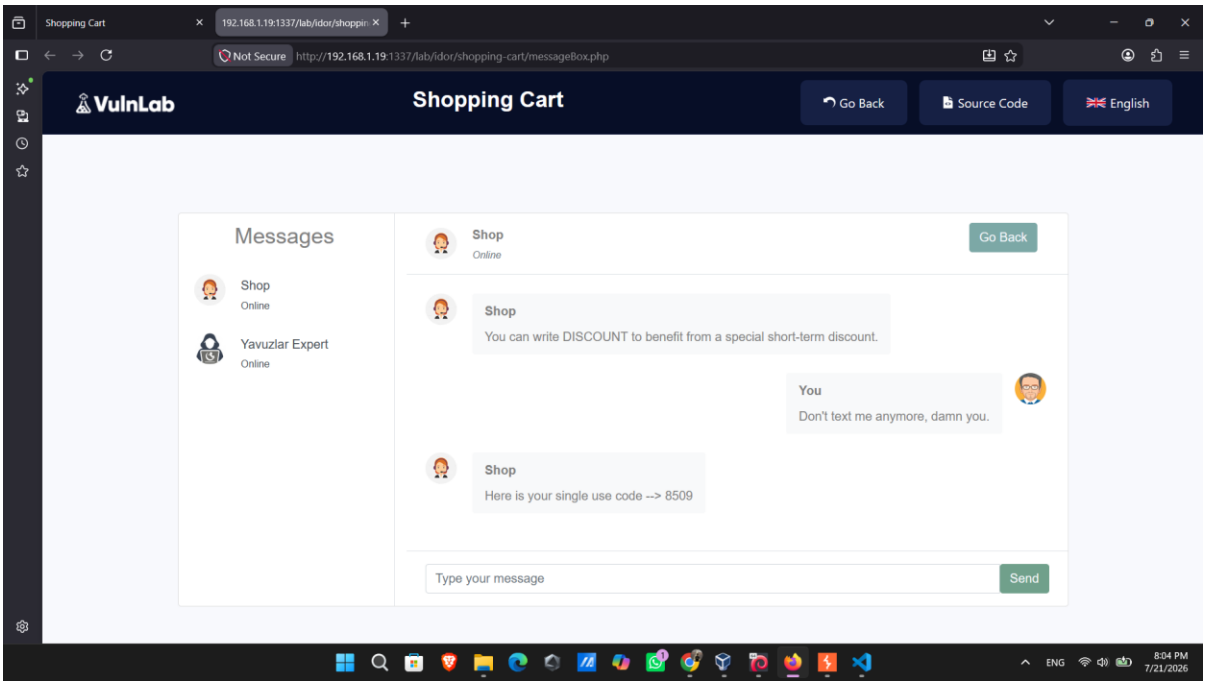


*Figure 4. Message box containing the OTP to be submitted.*

**Request**

Pretty Raw Hex \n

```
POST /lab/idor/shopping-cart/code.php?hesoyam=JGJhbGFuY2U9MTAw HTTP/1.1
Host: 192.168.1.19:1337
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:152.0) Gecko/20100101
Firefox/152.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Content-Type: application/x-www-form-urlencoded
Content-Length: 14
Origin: http://192.168.1.19:1337
Connection: keep-alive
Referer:
http://192.168.1.19:1337/lab/idor/shopping-cart/3Dvalid.php?code=SSBoZWFyZCB0aG
UgYWRtaW4gaXMgZm9yZ2V0ZnVs
Upgrade-Insecure-Requests: 1
Priority: u=0, i

inputCode=8509
```

*Figure 5. Intercepted OTP submission request.*

The request used to submit the OTP contained an additional parameter named "hesoyam", also Base64-encoded. Decoding this parameter revealed the value "$balance=100" — confirming that the application

transmits the user's account balance state through a client-controlled, encoded parameter rather than keeping it strictly server-side.

Step 5 — After submitting the OTP, the purchase completed successfully.

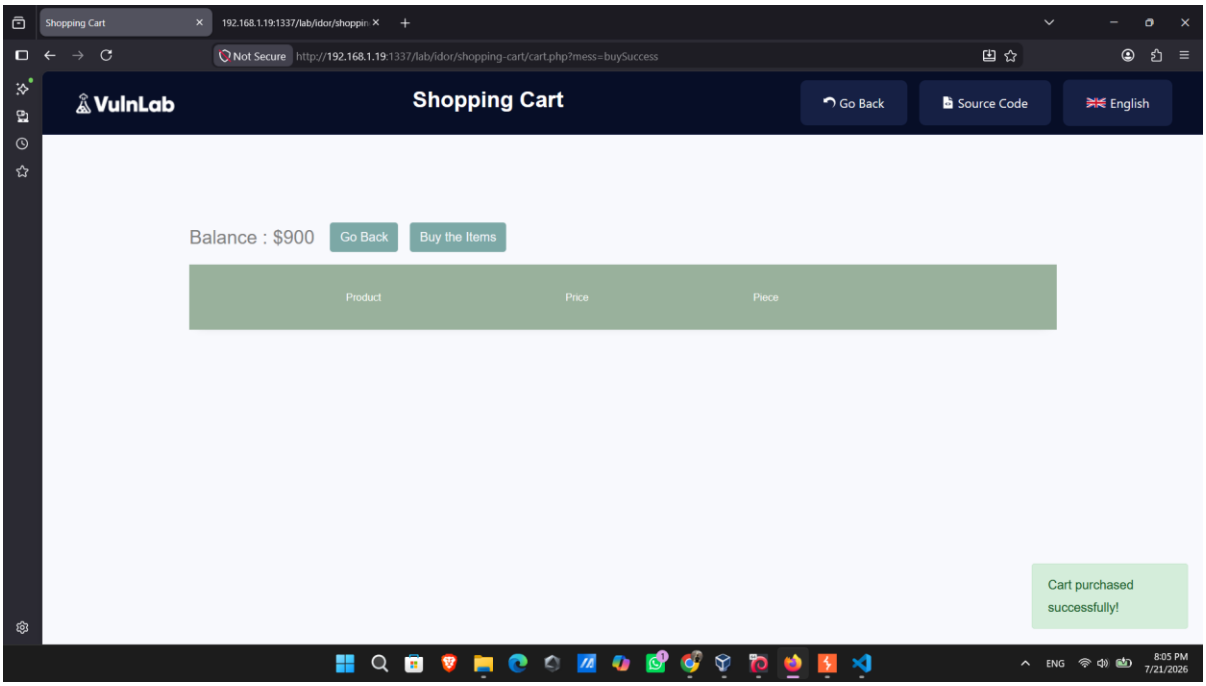


*Figure 6. Successful purchase confirmation.*

## 4.3 Identifying a Static / Predictable Transaction Reference

Step 6 — The tester repeated the purchase of the "Thief Cat" item and intercepted the request again.

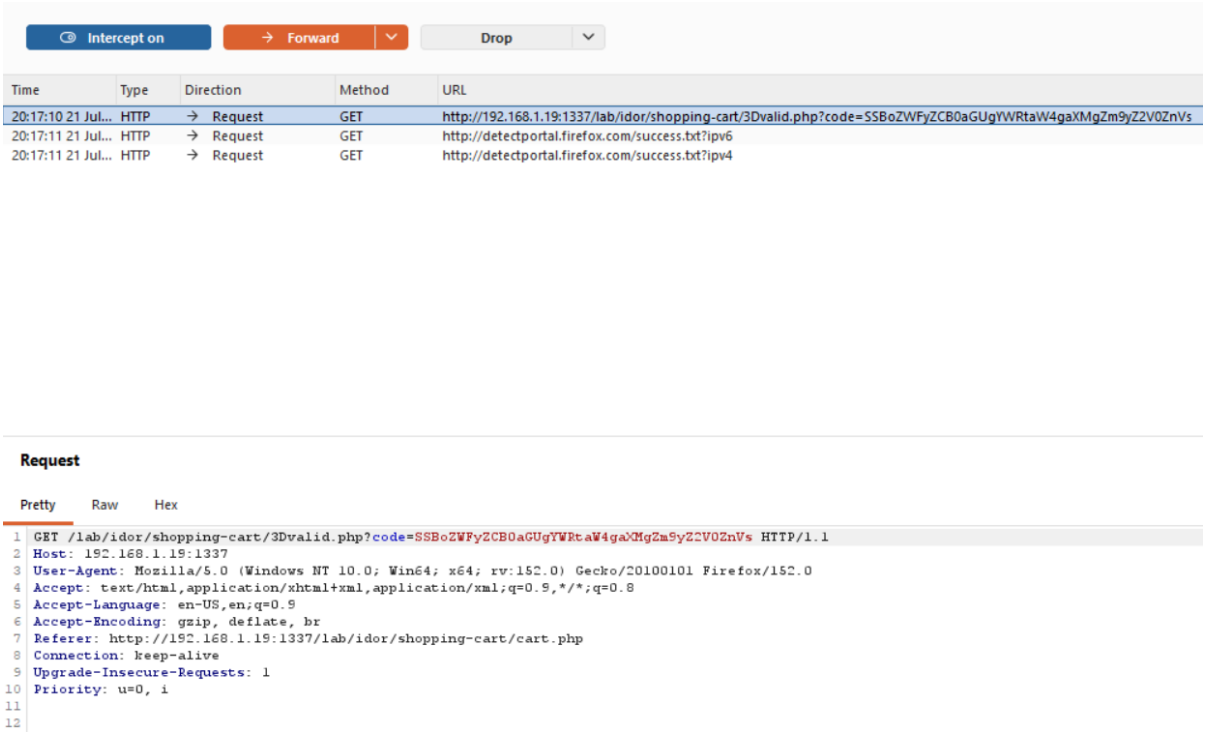


*Figure 7. Second purchase attempt, intercepted in Burp Suite.*

Critically, the OTP validation request generated for this second, independent transaction contained the exact same "code" parameter value observed in the very first purchase. This indicates that the "code" value is not randomly generated per transaction or per session — it is a static, predictable reference that can be reused.

Step 7 — The tester intercepted the OTP submission again for this second transaction.

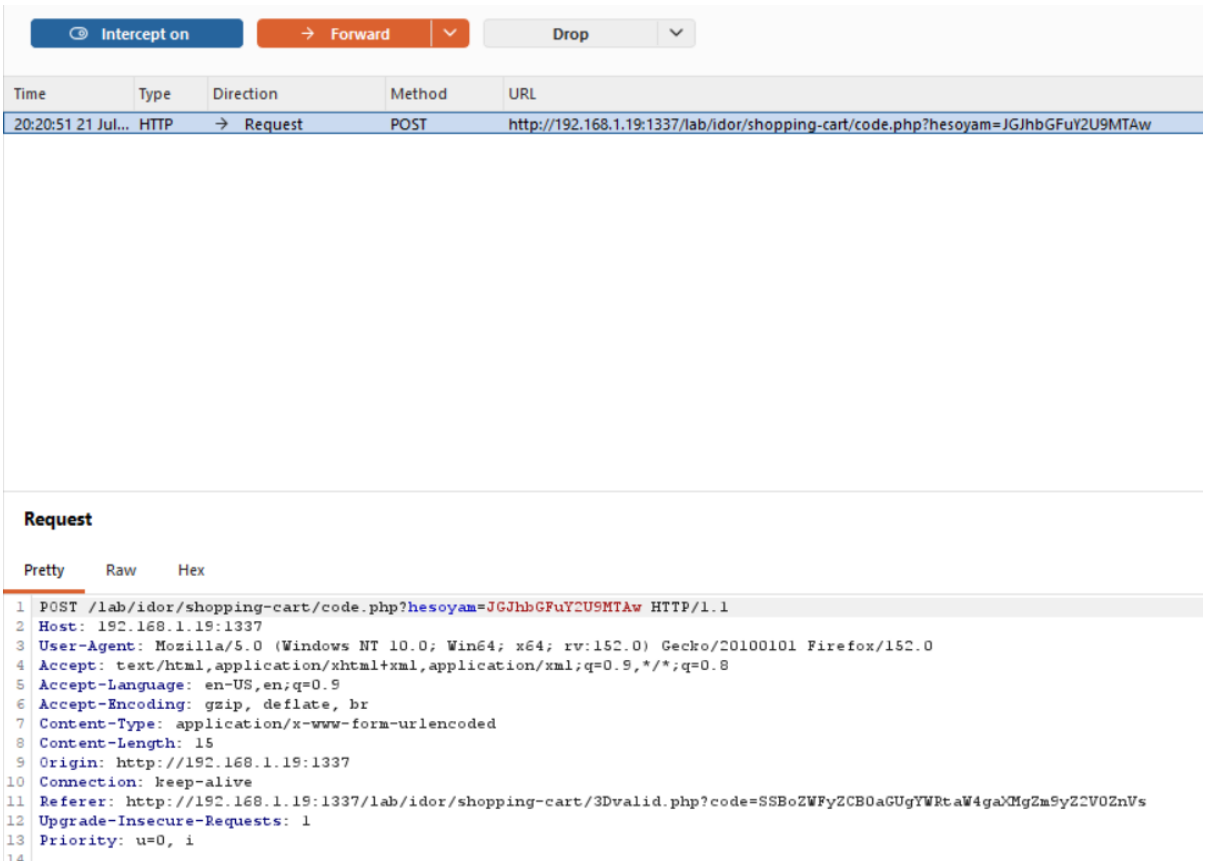


*Figure 8. Second OTP submission request, intercepted.*

The "hesoyam" parameter again decoded to the same value, "$balance=100", confirming this value is also static across separate transactions rather than reflecting a dynamically-tracked, server-side transaction state.

As a first tampering attempt, the tester modified the decoded balance value to "1000", re-encoded it to Base64, and forwarded the modified request to test whether the server trusted the client-supplied balance.

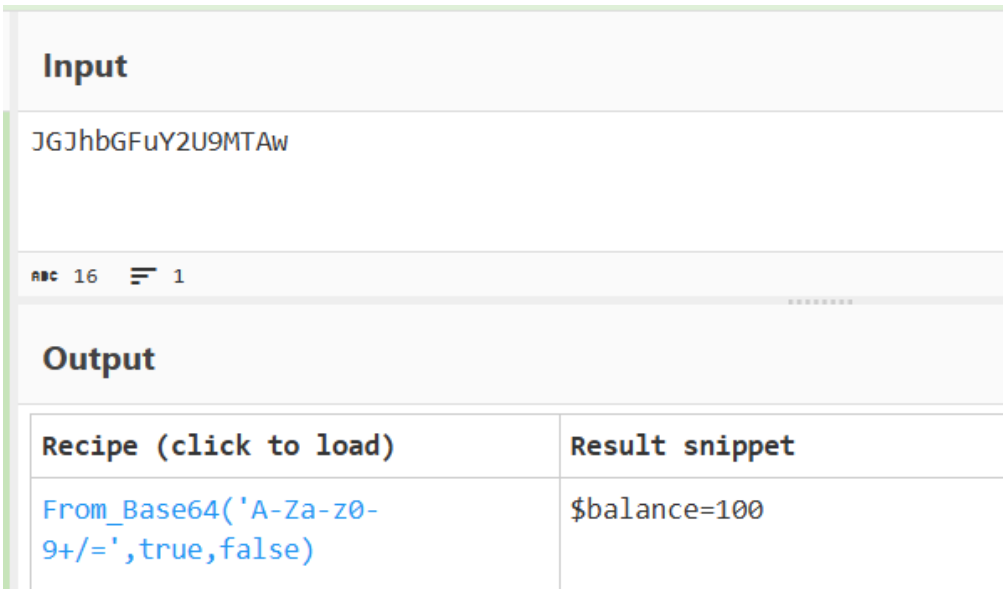


*Figure 9. Attempting to modify the encoded balance parameter to 1000.*

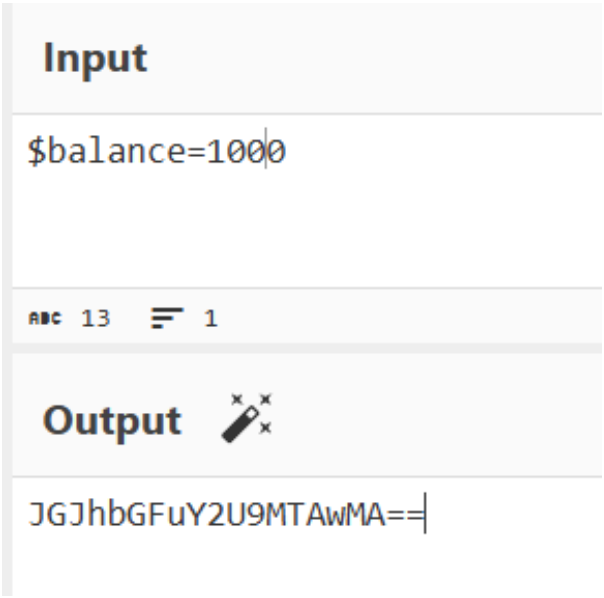


*Figure 10. Forwarding the modified request in Burp Suite.*

This particular tampering attempt had no effect — the purchase proceeded normally as if the parameter had not been modified, indicating the server does not actually trust this specific client-supplied value for the balance calculation itself.

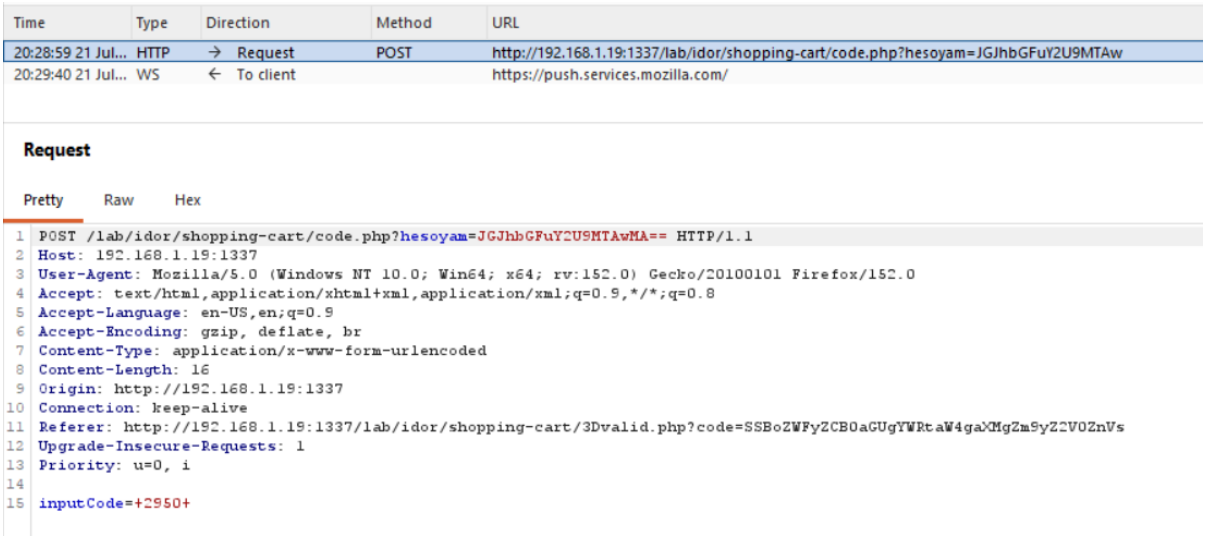


*Figure 11. Purchase result after the failed balance-tampering attempt.*

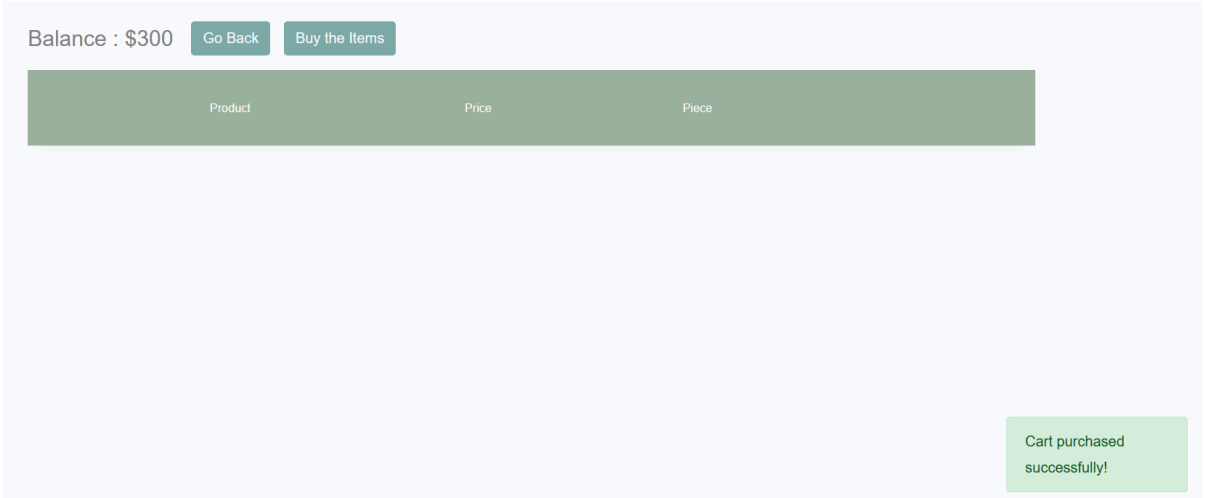


*Figure 12. Application state remaining unaffected by the tampered parameter.*

## 4.4 Bypassing the Balance Check via Direct Endpoint Access

Step 8 — Since both purchases required OTP validation using the exact same "code" reference value, the tester hypothesized that the OTP/transaction validation page could be reached directly, without going through the "Buy Item" step where the balance check is enforced. The tester constructed the following direct request:

```
GET /.../valid.php?code=SSBoZWFyZCB0aGUgYWRtaW4gaXMgZm9yZ2V0ZnVs
```

(where the "code" value is the same static, Base64-encoded reference — "I heard the admin is forgetful" — observed in every prior transaction)

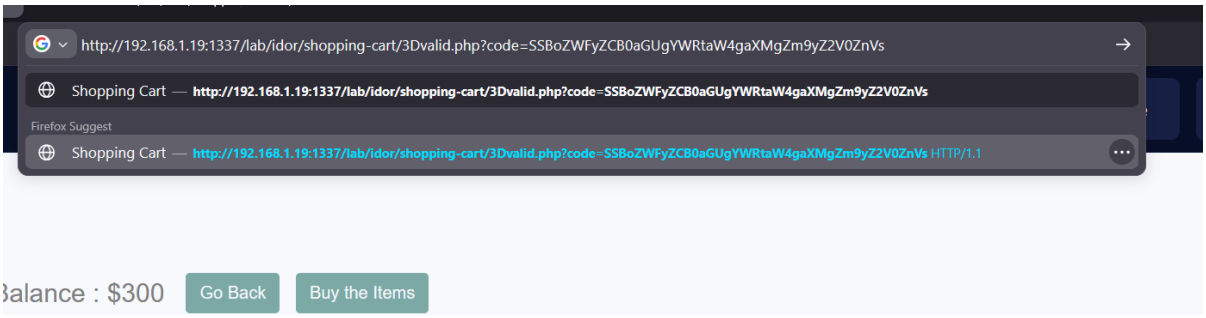


*Figure 13. Directly requesting the OTP validation endpoint, bypassing "Buy Item".*

The result confirmed the vulnerability hypothesis: the OTP validation page loaded successfully and accepted the request, even though no legitimate "Buy Item" / balance-check request had been made beforehand for this attempt.

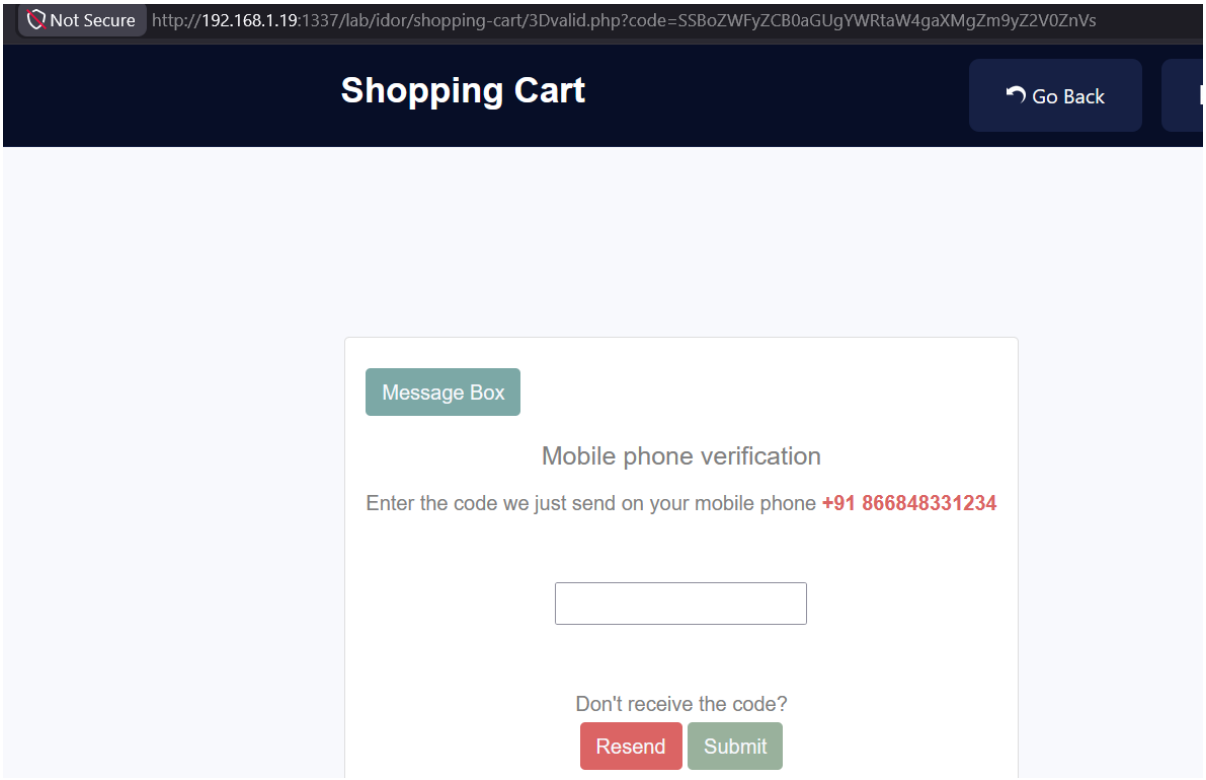


*Figure 14. The OTP validation page rendering successfully from a direct, out-of-sequence request.*

This demonstrates that the endpoint performs no server-side check to confirm that the request is tied to a previously initiated, authorized, and balance-validated transaction for the current user — a clear Insecure Direct Object Reference / broken workflow enforcement condition.

## 4.5 Confirming Impact: Purchasing an Item Above the Available Balance

Step 9 — To confirm real-world impact, the tester attempted to purchase an item named "Finish Flag", priced at $1049.99 — an amount well above the account's actual balance.

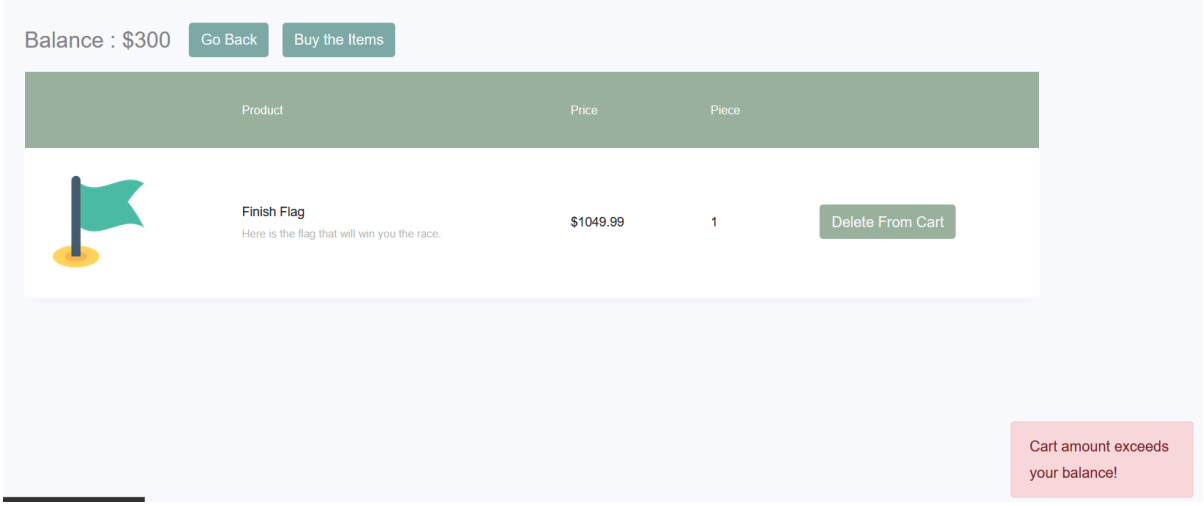


*Figure 15. Attempting to buy "Finish Flag" ($1049.99) through the normal flow, which correctly returns an insufficient-balance error.*

As expected, using the normal "Buy Item" flow correctly triggered an "insufficient balance" error, since the client-facing balance check functioned as intended at that step.

However, instead of clicking "Buy Item", the tester directly navigated to the OTP validation URL — bypassing the balance-check step entirely.

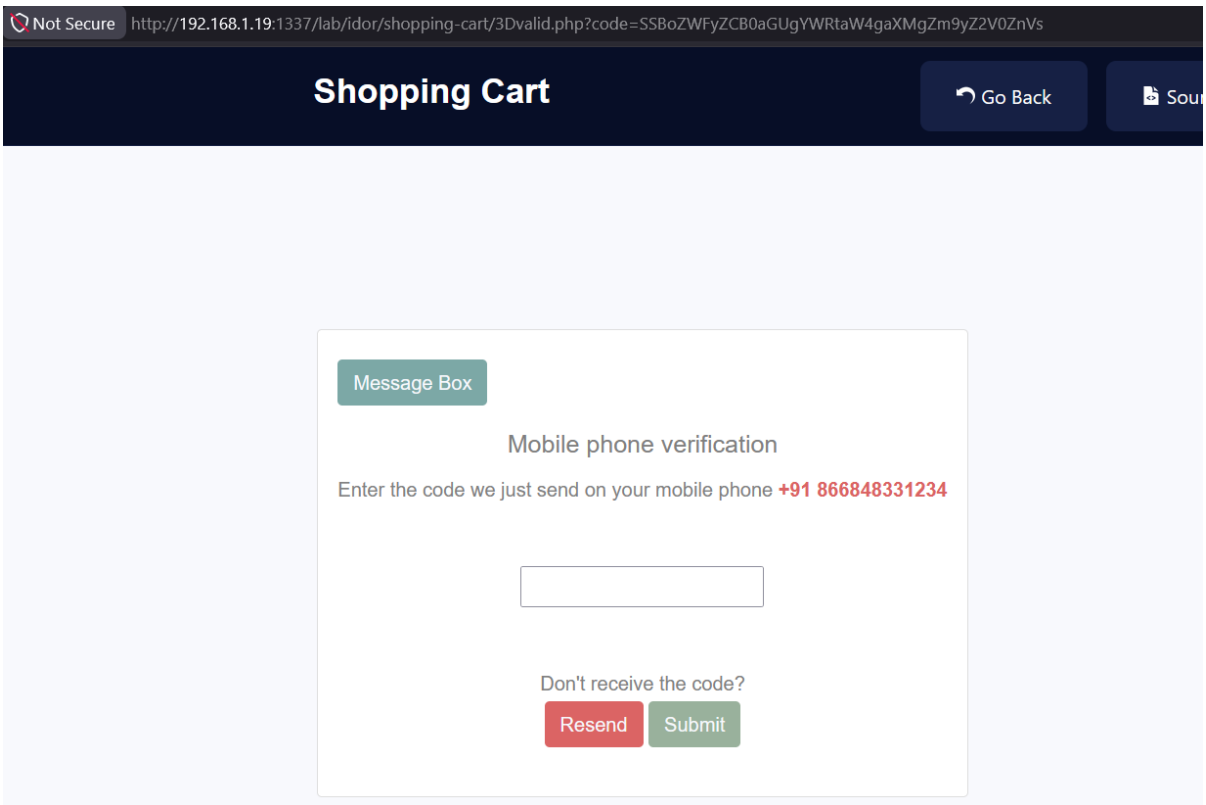


*Figure 16. Bypassing "Buy Item" by requesting the OTP validation endpoint directly for the "Finish Flag" purchase.*

The bypass succeeded: the OTP validation page loaded and prompted for an OTP, despite the account having insufficient funds for this item.

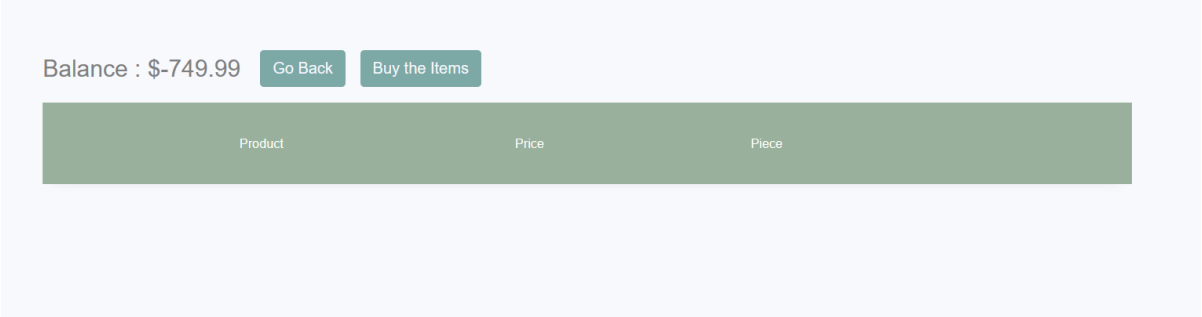


*Figure 17. OTP submitted successfully, completing the purchase of "Finish Flag" despite insufficient balance.*

**The purchase completed successfully. This conclusively demonstrates that the balance/funds validation control can be fully bypassed by directly invoking the OTP validation endpoint using its static, guessable reference value — allowing any authenticated user to obtain items regardless of their actual account balance.**

## 5. Root Cause Analysis

The underlying weaknesses that make this vulnerability possible are:

**1.** Balance validation is enforced only once, at the client-initiated "Buy Item" step, and is never re-checked at the final transaction-completion step (OTP validation). This is a classic Time-of-Check to Time-of-Use (TOCTOU)-style workflow flaw at the business-logic level.

**2.** The OTP/transaction validation endpoint is directly reachable via URL and uses a static, predictable "code" reference value ("I heard the admin is forgetful", Base64-encoded) instead of a cryptographically random, single-use, per-transaction, per-user token that is generated and tracked server-side.

**3.** There is no server-side session or transaction-state enforcement ensuring that the OTP validation step can only be reached after a legitimate, previously-created, balance-checked transaction record exists for the current user. In other words, step 2 of the workflow does not verify that step 1 actually happened.

**4.** Sensitive transaction state — including the account balance itself ("hesoyam" = "$balance=100") — is passed to and received back from the client via an encoded parameter. Base64 is an encoding scheme, not encryption; it provides no integrity or confidentiality guarantee and can trivially be decoded, inspected, and (in a differently-implemented version of this same flaw) tampered with by any user.

**5.** Taken together, points 1–3 constitute an Insecure Direct Object Reference: the validation endpoint is a directly-addressable "object" (a pending transaction) that is accessed using a key ("code") that is not unique, not unpredictable, and not authorization-checked against the requesting user's own legitimate transaction state.

## 6. CVSS v3.1 Severity Scoring

**Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N  Base Score: 7.5 (Medium)**

| Metric | Value | Justification |
|---|---|---|
| Attack Vector (AV) | Network (N) | The endpoint is reachable over the web application via HTTP requests. |

| | | |
|---|---|---|
| Attack Complexity (AC) | Low (L) | No special conditions are required; the attacker only needs to change the request URL/parameter. |
| Privileges Required (PR) | Low (L) | A standard authenticated user account with shopping access is sufficient. |
| User Interaction (UI) | None (N) | No action from another user or victim is required. |
| Scope (S) | Unchanged (U) | The impact is confined to the vulnerable component/application itself. |
| Confidentiality (C) | None (N) | No sensitive data disclosure results directly from the flaw. |
| Integrity (I) | High (I:H) | An attacker can complete a purchase transaction while bypassing the balance/funds validation control, directly corrupting the integrity of the transaction/business logic. |
| Availability (A) | None (N) | The flaw does not impact application availability. |

Note on Scope and Integrity impact: although this finding does not disclose confidential data (C:N) or affect availability (A:N), it was scored I:H because it allows an attacker to corrupt the intended state of a core business function — completing a paid transaction without possessing sufficient funds — which is a high-integrity impact on the application's transactional data and business logic, even though it is confined to the application itself (S:U).

## 7. Business Impact

- Direct financial loss: users could obtain paid items/services without actually having sufficient balance, resulting in unearned goods or services.
- Erosion of trust in the platform's transaction integrity, since the "insufficient balance" control can be silently bypassed.
- Potential for abuse at scale: because the bypass only requires replaying a static, known URL/parameter, it could be scripted and repeated by any authenticated user without needing to discover a new value per attempt.
- Reputational and compliance risk if this pattern exists in a production e-commerce or fintech-style application, since it directly affects payment/balance integrity controls.

## 8. Remediation Recommendations

**1.** Re-validate the account balance/funds server-side at every stage of the transaction, especially immediately before the purchase is finalized (i.e., before OTP acceptance completes the transaction) — not only at the initial "Buy Item" step.

**2.** Replace the static "code" value with a cryptographically random, unpredictable, single-use token generated per transaction and per user. Store this token server-side, bound to the specific transaction record, and invalidate it immediately after use or after a short expiry window.

**3.** Enforce strict workflow/state validation: the OTP validation endpoint must verify that a corresponding, legitimate, still-valid, balance-checked transaction record exists for the authenticated

user before accepting any OTP input. Direct or out-of-sequence access to this endpoint without a valid pending transaction must be rejected.

**4.** Apply object-level authorization checks (ownership validation) on every transaction reference used in a request — confirm the pending transaction referenced by the token belongs to the currently authenticated user before processing it. This is the standard fix pattern for IDOR.

**5.** Never transmit sensitive server-side state (such as account balance) to the client and trust it back unmodified. Keep balance and transaction state exclusively on the server side, associated with the user's session, and recompute/re-verify it from the source of truth (the database) at each step.

**6.** Do not rely on Base64 (or any reversible encoding) as a security control. If any transaction state must pass through the client, use a signed and/or encrypted token (e.g., a server-signed JWT with short expiry) so tampering can be detected and rejected.

**7.** Add server-side logging, monitoring, and rate-limiting on the OTP/transaction validation endpoint to detect and throttle anomalous direct-access patterns (e.g., repeated identical "code" values across different users or transactions).

**8.** Include this specific bypass scenario (skipping "Buy Item" and going straight to OTP validation) in regression/security test suites going forward.

## 9. Conclusion & Key Takeaways

This exercise reinforced several important lessons about Insecure Direct Object Reference and business-logic testing beyond the textbook "change the ID in the URL" example:

- IDOR is not limited to obvious identifiers like numeric IDs in a URL — any reference value used to access or resume a workflow step (such as a transaction/OTP validation token) can be an IDOR vector if it is static, predictable, or not authorization-checked.
- Multi-step workflows (e.g., "initiate purchase → validate OTP → complete purchase") must enforce state/sequence server-side at every step; a security control (like a balance check) that is only enforced at one step in the flow can be trivially bypassed by skipping directly to a later step.
- Encoding (Base64) is not a substitute for access control or encryption — decoding parameters during interception is a simple but highly effective technique for uncovering hidden application logic and state.
- Testing the same flow twice and comparing the requests (as done in Steps 6–7 above) is a practical way to distinguish genuinely dynamic, per-transaction values from static ones that may be reusable or predictable — a useful general heuristic when hunting for IDOR.

*Overall, this finding demonstrates a practical, end-to-end IDOR / broken workflow vulnerability: from initial reconnaissance of the purchase flow, through parameter analysis and decoding, to a fully confirmed proof-of-concept that bypasses a core financial control. It is documented here as part of ongoing self-study in web application penetration testing.*